Efficient Remote Sensing Instance Segmentation With Linear-Time State-Space-Distilled Visual Foundation Models

Qinzhe Yang, Keyan Chen, Jia Xu, Zhenwei Shi, *Senior Member, IEEE*, and Zhengxia Zou, *Senior Member, IEEE*

***Abstract*—The computational complexity of Transformers scales quadratically with the number of tokens, which significantly constrains the efficiency of vision models, particularly recent ViT-based foundation models in dense prediction tasks. The instance segmentation, a typical dense visual prediction task in the remote sensing field, faces similar challenges. In this article, inspired by the recent advances of knowledge distillation in large language models, we introduce remote sensing instance segmentation with linear-time state space distilled (RS4D)—a new remote sensing instance segmentation method with linear computational complexity, which addresses the inefficiency of long-sequence modeling through the distilled state-space modeling. We propose an adaptive noise and masking knowledge distillation training method for pretraining lightweight state-space model (SSM) backbones, which effectively compresses knowledge from the vast self-attention space into a compact, dense linear state space. We also design a remote sensing image instance segmentation architecture based on this lightweight visual encoder, where we explore variants of three different backbones and two segmentation heads. Extensive experiments are conducted on multiple benchmark datasets, including SSDD, WHU, and NWPU. Compared with ViT-based approaches, our proposed SSM backbone achieves an $\times 8$ reduction in parameters and a $\times 9$ reduction in floating-point operations (FLOPs) while maintaining comparable or superior accuracy to both ViT- and CNN-based instance segmentation methods. The implementation codes have been publicly available at https://github.com/QinzheYang/RS4D**

***Index Terms*—Knowledge distillation, remote sensing, state-space model, visual foundation model.**

I. Introduction

THE advancement of deep learning models has accelerated the development of instance segmentation methods [1], [2], [3], [4], [5]. Initially, these methods primarily relied on convolutional neural network (CNN) architectures [6], [7], [8], [9], [10], [11], [12]. With the emergence of visual foundation models, Transformer-based architectures have gained prominence in image segmentation tasks, exemplified by the segment anything model (SAM) [13]. These approaches have been successfully extended to instance segmentation in remote sensing applications [14], [15]. They often incorporate pretrained foundation models, such as Vision Transformer (ViT) [16], as their backbone architecture. While these models demonstrate superior accuracy compared to traditional CNN-based approaches, they require significantly more parameters and computational resources. This is because dense prediction typically requires processing high-resolution feature maps and more tokens, so the quadratic self-attention cost in ViTs leads to rapidly increasing computation and memory usage as resolution grows.

Remote sensing image instance segmentation fundamentally operates as a dense prediction task, which manifests as a long-sequence prediction problem [2], [14], [17], [18], [19], [20]. For example, processing a 1024 $\times$ 1024 pixel input image with a spatial resolution of 16 $\times$ 16 pixels per token generates a sequence length of 4096 tokens. However, the self-attention mechanism in Transformer architectures exhibits quadratic computational complexity with respect to sequence length [21], [22], [23], [24], [25], [26], [27], [28]. This computational burden leads to substantial memory requirements and slow inference speeds when applying Transformer models to remote sensing instance segmentation tasks. Consequently, these models prove impractical for time-sensitive applications or resource-constrained edge devices, particularly in scenarios such as in-orbit processing.

The state-space model (SSM) [29], [30], [31], [32], [33] has emerged as a promising framework for processing long sequences, gaining significant attention due to its linear computational complexity. In processing sequential data, the model's state and output equations are interwoven in a way that mathematically unfolds into a structure analogous to 1-D convolution operations. This way of mathematical modeling enables efficient computation and makes SSM particularly well-suited for dense prediction tasks, such as instance segmentation. Compared with Transformer-based architectures, SSM offers enhanced performance potential and optimization capabilities. In particular, the linear-time token mixing in SSMs enables global context modeling on long sequences

Received 30 December 2025; revised 24 March 2026; accepted 11 May 2026. Date of publication 22 May 2026; date of current version 15 June 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62125102, Grant 62471014, Grant U24B20177, Grant U25A20401, and Grant 623B2013; and in part by the Fundamental Research Funds for the Central Universities. *(Qinzhe Yang and Keyan Chen contributed equally to this work.) (Corresponding author: Zhengxia Zou.)*

Qinzhe Yang is with the Shen Yuan Honors College, Beihang University, Beijing 100191, China.

Keyan Chen, Zhenwei Shi, and Zhengxia Zou are with the Department of Aerospace Intelligent Science and Technology, School of Astronautics, and the State Key Laboratory of Virtual Reality Technology and Systems, Beihang University, Beijing 100191, China (e-mail: zhengxiazou@buaa.edu.cn).

Jia Xu is with the Qian Xuesen Laboratory of Space Technology, China Academy of Space Technology, Beijing 100094, China.

Digital Object Identifier 10.1109/TGRS.2026.3696104

1558-0644 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

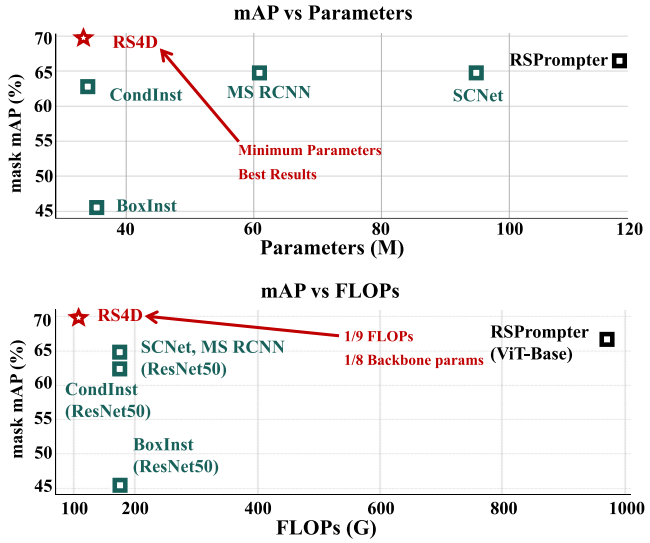


Fig. 1. Comparison with state-of-the-art methods on the SSDD dataset evaluating mask mAP, model parameters, and FLOPs.

at substantially lower memory/computation than quadratic attention, which can translate into a better accuracy–efficiency tradeoff for high-resolution dense prediction when paired with appropriate architectural design.

In SSM instance segmentation, there have already been many outstanding works. MambaInst [34] achieves efficient instance segmentation by designing the LightSSM Block and FRDown module, maintaining linear computational complexity. Spatial-Mamba [35] tends to enhance the spatial relationship capture of SSM through convolution. However, this article aimed at presenting a novel lightweight visual pretraining approach for ViTs utilizing knowledge distillation to compress information from extensive self-attention spaces into compact linear state spaces. We name our method remote sensing instance segmentation with linear-time state space distilled (RS4D) visual foundation models. In recent research on large language models, especially in reasoning models, model distillation has proven to be an effective lightweight strategy and has demonstrated significant potential in architectures like Llama, DeepSeek, and others [36], [37]. We develop a new visual encoder network architecture leveraging SSMs that efficiently encodes complex knowledge with minimal parametric overhead. Our framework encompasses multiple architectural variations, including three distinct SSM backbones and two instance segmentation heads, each demonstrating robust performance. Comprehensive evaluations on established benchmark datasets (SSDD [38], WHU [39], and NWPU [40]) validate our approach, with results presented in Fig. 1. The proposed RS4D architecture achieves significant computational efficiency, reducing floating-point operations (FLOPs) by $\times 9$ compared with ViT-based methods and by $\times 6$ relative to ConvNet approaches, while maintaining comparable accuracy. Most notably, RS4D matches the performance metrics of RSPrompter [14] (with pretrained ViT-based encoder) while requiring only 1/4 of the model parameters and GPU memory during training operations.

The contributions of this article can be summarized as follows.

1) We present a novel remote-sensing-oriented instance segmentation architecture optimized for dense prediction, leveraging SSMs. The proposed approach combines an efficient scanning mechanism with knowledge distillation techniques derived from large-scale ViTs, thereby achieving both architectural efficiency and effective compression of foundational model knowledge.
2) We systematically investigate various architectural configurations, implementing three distinct SSM backbones and two instance segmentation heads. Through this exploration, we identify design structures that are more suitable for remote sensing dense prediction tasks than existing methods.
3) Comprehensive experiments conducted on the SSDD, WHU, and NWPU datasets reveal that our model achieves performance parity or superiority compared with existing CNN- and ViT-based instance segmentation methods, while significantly reducing computational demands. Specifically, the model requires only 1/8 of the parameters and 1/9 of the FLOPs relative to baseline architectures.

II. Related Works

This section reviews four key areas relevant to our proposed method: instance segmentation, visual foundation models, lightweight models, and state-space models.

A. Instance Segmentation

The instance segmentation is a fundamental computer vision task that integrates pixel-level semantic segmentation with instance-level object detection. The methods can be classified into two primary categories: two-stage and one-stage methods. Representative two-stage methods include Mask R-CNN [41], MS R-CNN [42], HTC [43], and HQ-ISNet [10], while one-stage methods include YOLACT [6], PolarMask [8], CondInst [44], SOLOv2 [45], and MaskFormer [46]. The difference between the two methods is that the one-stage method directly generates predictions from the image, while the two-stage method first generates candidate regions and then performs classification and regression. Two-stage methods typically follow a proposal-and-refine pipeline, often achieving strong accuracy but with extra cost from proposal and RoI processing. In contrast, one-stage methods predict masks directly from dense features without explicit RoI proposals, usually offering higher efficiency at the cost of more careful design to match two-stage quality.

Recent advances in foundation models have catalyzed significant progress in segmentation tasks. In particular, prompt-based segmentation methods, exemplified by the SAM, have emerged as versatile solutions for diverse segmentation challenges [13], [47]. The integration of large language models has opened new avenues in language-guided segmentation research [48]. Furthermore, researchers have successfully adapted the denoising diffusion process, inherent in diffusion

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

models, to segmentation applications [49], [50]. Recent investigations have also demonstrated the potential of zero-shot segmentation by leveraging segmentation knowledge extracted from pretrained foundation models [51], [52].

Remote sensing instance segmentation poses additional challenges compared with natural-image benchmarks, including very high spatial resolution, small and sparsely distributed targets, and modality variations [2]. To address these characteristics, several methods have been proposed specifically for remote sensing imagery, such as HQ-ISNet [10] and CATNet [53]. In this article, we include these RS-oriented methods as representative remote-sensing baselines and highlight how our approach differs by transferring foundation-model segmentation priors into a lightweight linear-time SSM backbone.

B. Visual Foundation Models

SAM exemplifies visual foundation models specialized in image segmentation [13]. Its architecture comprises three key components: a pretrained image encoder, a prompt encoder, and a decoder. The image encoder converts input images into deep feature representations, while the prompt encoder transforms user-defined inputs (target points, masks, and boxes) into embeddings. The decoder then synthesizes outputs from both encoders to generate foreground masks at specified locations. SAM has demonstrated exceptional performance across various general vision applications and exhibits robust generalization capabilities [14], [54], spurring extensive research and numerous task-specific variants [47], [55], [56].

Building on SAM, several new image segmentation methods have been introduced. RSPrompter [14], which implements an automatic prompter by utilizing the intermediate outputs from the image encoder to enable the automatic instance segmentation. MobileSAM [55] and EfficientSAM [47] focus on compressing the image encoder through distillation into a smaller ViT. SlimSAM [57] combines pruning and distillation techniques to develop a lightweight SAM variant. Despite these advancements, these approaches encounter a fundamental limitation: their performance and computational requirements remain intrinsically bound to the transformer architecture. This inherent dependency creates a significant challenge in optimizing the tradeoff between processing efficiency and segmentation accuracy, making it difficult to achieve an optimal balance between computational speed and performance quality.

Beyond general-purpose promptable segmentation models such as SAM, recent studies have explored adapting SAM-like priors to remote sensing imagery via prompt generation, domain-aware fine-tuning, and RS-specific prompting strategies. Representative directions include segmentation-guided SAM variants [58] and remote-sensing prompters that generate prompts from RS imagery [14]. These works suggest that foundation-model priors can benefit RS segmentation. However, most approaches still rely on computationally heavy backbones. Our work focuses on distilling SAM's segmentation priors into a compact student backbone with linear-time complexity.

C. Lightweight Models

The lightweight deep learning methods can be categorized into five main groups [59], [60], [61], [62]: knowledge distillation, model pruning, quantization, low-rank decomposition, and lightweight model design. Knowledge distillation transfers computational knowledge from a complex teacher model to a more compact student model [16], [59], [63], [64]. Network pruning enhances model efficiency by eliminating redundant or less significant weights [60], [64], [65], [66], effectively reducing the network's complexity. Model quantization research primarily focuses on two directions: multinetwork weight sharing and single-node weight reduction [60], [67], [68], [69]. Low-rank decomposition techniques, such as SVD and PQ, decompose the model's original tensor weights into multiple low-rank tensors [61]. Unlike these compression strategies, the lightweight design emphasizes building efficient network architectures from inception, typically incorporating lightweight CNN architectures and optimized neural operators [62], [70], [71], [72].

While previous approaches have predominantly centered on convolutional or attention-based network designs, our research introduces a novel perspective by employing a visual state-space model. This approach facilitates knowledge transfer from complex, attention-mechanism-based foundation models to a more efficient state-space model network.

D. State-Space Models

The SSM refers to a class of mathematical frameworks for representing discrete dynamical systems that describe both the system's temporal evolution and the relationship between hidden states and observable data. Originally developed in control theory, SSM processes continuous sequences to generate predictive outputs. The model is characterized by two fundamental equations: the state equation, which defines the relationship between inputs and previous states through matrix operations, and the output equation, which determines how the state matrix is transformed into observable outputs

$$\begin{aligned} h_k &= Ah_{k-1} + Bx_k \\ y_k &= Ch_k \end{aligned} \tag{1}$$

where $h_k \in \mathbb{R}^{n\times l}$ represents the hidden state at step k. $h_{k-1} \in \mathbb{R}^{n\times l}$ denotes the hidden state at step $k-1$, $A \in \mathbb{R}^{n\times n}$. $B \in \mathbb{R}^{n\times d}$, and $C \in \mathbb{R}^{n\times n}$ are the parameter matrices. $x_k \in \mathbb{R}^{d\times l}$ and $y_k \in \mathbb{R}^{n\times l}$ represent the input and output at step k, respectively. l, d, and n denote the sequence length, input vector dimension, and state size, respectively.

Recently, several significant developments have been made in SSM, aimed at enhancing their application in neural networks. Gu et al. [33] introduced the time-step memory mechanism and HiPPO, leading to the development of the S4 model [30]. This addressed two major challenges in sequence modeling: the computational complexity associated with Transformers [73], [74] and the sequence forgetting problem inherent in RNNs [75]. Building upon insights from S5 [76], researchers proposed Mamba [77], which incorporates a hardware-aware algorithm and introduces a structured SSM gating mechanism [29], [31], [77]. Subsequently, two

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

major architectural improvements emerged: Vision Mamba [31], which focuses on structure blending, and VMamba [78], which implements SS2D scanning. These innovations have established two primary directions for researchers working on vision-related tasks [29], [32], [79].

In recent years, the Mamba architecture has emerged as a promising alternative to Transformers, offering reduced computational complexity through selective modeling and linear scaling. Mamba-based approaches have shown remarkable performance in various remote sensing tasks, including classification, semantic segmentation, change detection, and image super-resolution [80], [81], [82], [83]. Several methods have advanced this field: RSMamba [84] introduced a multidirectional scanning mechanism; RSCaMa [85] enhanced temporal change detection through cross-temporal scanning; ChangeMamba [86] combined global and local modeling approaches; RS3Mamba [87] explored local detail enhancement within the SSM; and Pan-Mamba [88] advanced pan-sharpening technology while reducing redundancy. Despite these efforts, the research in instance segmentation remains limited. For example, MambaRSIS [89] harnesses contextual information to facilitate multiscale feature aggregation while investigating SSM blocks for efficient remote sensing image segmentation. The results are promising, demonstrating the potential of SSMs in remote sensing applications. Our work focuses on transferring segmentation priors from a promptable foundation model to a lightweight SSM student via knowledge distillation, and systematically studies RS-suitable design choices for efficient instance segmentation. To address this, we propose RS4D, a method specifically designed for efficient instance segmentation in remote sensing applications.

III. Methodology

This section introduces two key components of the proposed framework: first, the knowledge distillation mechanism that transfers pretrained knowledge from a visual foundation model to a lightweight SSM, and second, the efficient instance segmentation method designed for remote sensing imagery.

A. Linear-Time State-Space Knowledge Distillation

Our proposed method employs a knowledge distillation framework comprising a teacher model and a student model, with SSM as the core architecture. For the teacher model, we adopt the SAM Visual Encoder (ViT-Base), which has demonstrated superior feature extraction capabilities when trained on large-scale datasets [13]. The student model encompasses three novel architectures specifically designed to optimize inference speed, enhance learning robustness, and minimize storage requirements while effectively distilling knowledge from the SAM-Encoder. These lightweight SSM-based architectures achieve an optimal balance between computational efficiency and model performance.

1) SSM Student Architectures: Our framework implements three distinct lightweight SSM student architectures: VanillaMamba, TransMamba, and ScanningMamba, as illustrated in Fig. 2(c).

VanillaMamba consists of 12 stacked Vision Mamba blocks (ViM blocks). While this architecture achieves initial convergence during the first training stage, its simplified structure presents certain limitations. Notably, maintaining the original memory parameters while expanding the input resolution from 256 × 256 pixels to 1024 × 1024 pixels results in a ×16 increase in sequence length, potentially constraining the model's learning capacity.

The designed TransMamba employs a hybrid Mamba-Transformer architecture, as depicted in Fig. 3(a). The architecture consists of 12 layers, each containing a specialized TRVim block that integrates Mamba components with the Transformer layer. The TRVim block implements a sequential combination of a residual layer, normalization layer, vim layer, GELU activation function, and a multilayer perceptron (MLP). The integration with the Transformer architecture is achieved through a selective replacement of the attention layer, rather than substituting the entire stack. Although TransMamba exhibits enhanced learning capabilities compared to VanillaMamba, its increased parameter count leads to higher computational complexity. Consequently, it is the most computationally intensive among the three architectures, requiring longer inference times. Experimental results indicate that while the model successfully converges for bounding box (BBox) detection, its segmentation performance remains suboptimal. The computational process can be formally expressed as follows:

$$\begin{aligned} Y_{a-1} &= X_{a-1} + \text{TRVim}\left(\text{norm}\left(X_{a-1}\right)\right) \\ X_a &= Y_{a-1} + \text{linear}_{mlp}\left(\text{Gelu}\left(\text{norm}\left(Y_{a-1}\right)\right)\right) \end{aligned} \tag{2}$$

where $X_a \in \mathbb{R}^{b\times l\times d}$ denotes the intermediate state at layer a, TRVim$(\cdot)$ represents the Mamba block within TransMamba, $a \in [1, 12]$ represents the layer index, linear$_{mlp}(\cdot)$ signifies the MLP transformation, and Gelu$(\cdot)$ represents the GELU activation function.

ScanningMamba modifies the original Mamba architecture [77] by implementing both forward and backward scanning strategies. When processing an input sequence, the ScanningMamba block generates two versions: one in the original order and another in reverse order. These sequences are processed independently, and then, the reversed sequence is restored to its original order before being averaged with the forward-processed sequence. To better capture sequence complexity, we increased the number of stacking layers from 12 to 24. The detailed architecture is illustrated in Fig. 3(b). Despite its moderate size, ScanningMamba demonstrates robust learning capabilities and efficient inference. The process can be mathematically expressed as follows:

$$\begin{aligned} X_c^{\text{backward}} &= \text{Mambamixer}\left(\text{flip}\left(X_{c-1}\right)\right) \\ X_c^{\text{forward}} &= \text{Mambamixer}\left(X_{c-1}\right) \\ X_c &= X_{c-1} + \left(X_c^{\text{backward}} + \text{flip}\left(X_c^{\text{forward}}\right)\right)/2 \end{aligned} \tag{3}$$

where $X_c \in \mathbb{R}^{b\times l\times d}$ represents the output of layer c. $X_{c-1} \in \mathbb{R}^{b\times l\times d}$ denotes the output of layer $c-1$. $X_c^{\text{backward}} \in \mathbb{R}^{b\times l\times d}$ and $X_c^{\text{forward}} \in \mathbb{R}^{b\times l\times d}$ are intermediate variables in layer c. Mambamixer$(\cdot)$ represents the Mambamixer function. flip$(\cdot)$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

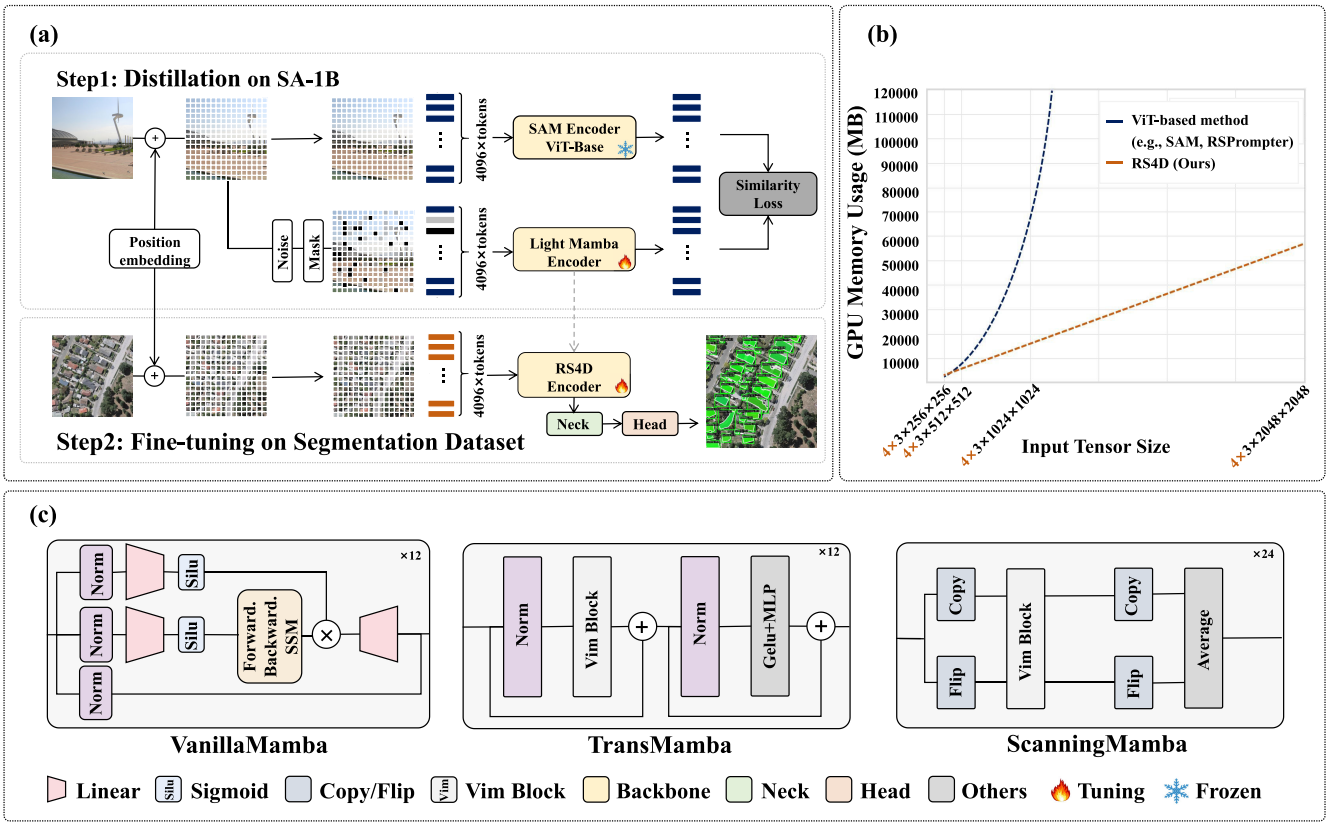


Fig. 2. Overview of our proposed RS4D. (a) Training process comprises two stages: 1) knowledge distillation from the ViT-based SAM encoder to our lightweight Mamba encoder, using SA-1B dataset [13] and 2) model fine-tuning on domain-specific remote sensing datasets. (b) In contrast to RSPrompter, our model achieves nearly linear computational complexity and reduces GPU memory consumption by 76% when processing 1024 × 1024 pixel images. (c) Exploration of multiple model implementations within the proposed framework, featuring three distinct backbone architectures.

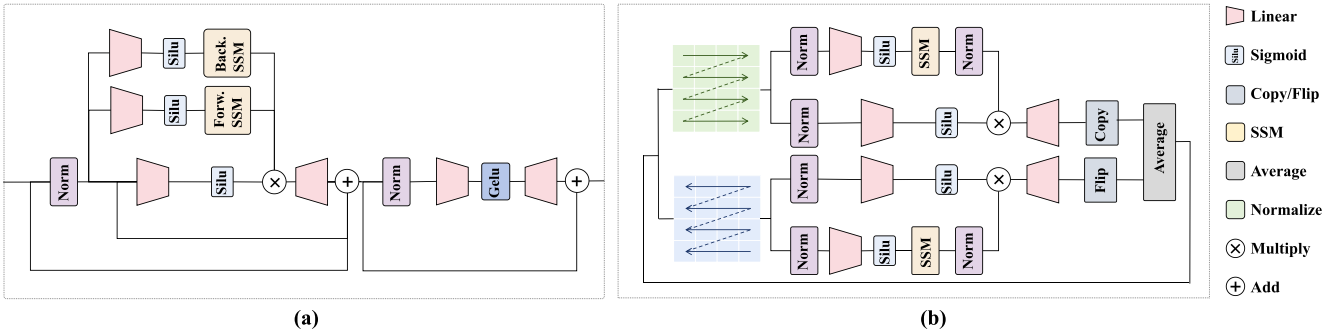


Fig. 3. Architectural details of (a) TransMamba and (b) ScanningMamba blocks.

denotes the sequence reversal function. c represents the layer number, ranging from 1 to 24.

Our three-student backbone design is guided by a core question: how to achieve linear-time computation suitable for high-resolution remote sensing instance segmentation while retaining global segmentation priors similar to SAM. Since SAM itself is a 12-layer transformer, the VanillaMamba merely substitutes the transformer with the capabilities of the SSM block.

In contrast, TransMamba explores whether combining the structures of SSM and transformers can complement the long-range modeling of SSM. One natural idea is to integrate SSM with self-attention mechanisms, leveraging the global associations of self-attention alongside the selective modeling of SSM. However, retaining self-attention would not alter the computational complexity of the model, which contradicts our lightweight design principles. Therefore, we removed the self-attention layers from the transformer framework and replaced them with SSM blocks, keeping the skip connections to prevent excessive information compression from SSM, which could lead to the loss of shallow-layer information. In addition, we replaced ReLU with GELU. Although ReLU was the original choice for the Transformer MLP, TransMamba is specifically designed for efficient and robust distillation in high-resolution remote sensing settings. This is particularly suitable for distillation in dense prediction scenarios. ReLU

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

has truncation at 0, which can lead to gradient clipping and activation sparsity, causing all negative responses to be clipped to 0. In contrast, GELU is a smooth gated activation function, resulting in a more continuous backpropagation process, which reduces training-induced jitter. In addition, many objects in remote sensing occupy only a tiny number of pixels in the overall high-resolution image, often accompanied by low-contrast complex backgrounds. Retaining some weak but useful texture or boundary cues can facilitate the refinement of small targets and boundary details. Therefore, we employ GELU in the lightweight MLP branch to enhance optimization robustness under masked and noise distillation inputs. It is important to note that in our mixed block, while the SSM path already provides a smooth gated nonlinear function (SiLU) intended to control information flow, we still need to offer a simple nonlinear supplement to encourage more stable activation of weak but important local responses, which is crucial in remote sensing images.

ScanningMamba tests whether there is a clear improvement in 2-D contextual completeness, specifically through feature aggregation via forward and backward scanning, which is a key factor for stable optimization and high mask quality. This design is motivated by the need for spatially consistent context aggregation in dense prediction tasks across the 2-D image plane. Pure 1-D ordering or unidirectional aggregation may introduce directional bias and incomplete context, which can adversely affect the consistency of masks in large high-resolution scenes. The memory mechanism of SSM is fundamentally based on its HIPPO matrix continuously compressing information. However, a single-path SSM inevitably biases the model toward remembering the most recently encountered information. Therefore, introducing multiple paths can effectively mitigate this limitation. ScanningMamba addresses this issue by employing bidirectional scanning on the 2-D token grid, achieving a more complete integration of spatial context while maintaining the linear time complexity, thus enhancing mask quality and optimization stability. On the other hand, introducing too many paths can substantially increase the computational burden on the model. For this reason, we designed ScanningMamba and expanded it to 24 layers based on our experimental results, as too few layers in SSM are insufficient to carry the prior information of SAM. We have validated this in subsequent experiments.

2) Distillation Process: During knowledge distillation, the student network is trained using unlabeled data through the following process. First, 50 000 augmented images (10k raw images) are fed into both the teacher and student models. We deliberately introduce random masks and noise to the student model's input pixels to improve the generalization of features via self-supervised learning. The noise amplitude is adaptively adjusted based on the maximum value of the output matrix to maintain stability

$$\begin{aligned} X' &= \text{mask}\left(X_{input}\right) + \alpha_e \times \epsilon \\ \alpha_e &= \min\left(\max_{b,l,d}\left(|Y^{e-1}|\right), \lambda_{const}\right) \\ \epsilon &\sim \text{Uniform}\,(0,1) \end{aligned} \tag{4}$$

where $X' \in \mathbb{R}^{b\times l\times d}$ denotes the student's inputs at the current epoch and $X_{input} \in \mathbb{R}^{b\times l\times d}$ represents the original image. The function mask$(\cdot)$ applies a binary mask to the input. $Y^{e-1} \in \mathbb{R}^{b\times l\times d}$ represents the student's output from the previous epoch, and $\max_{b,l,d}(|Y^{e-1}|)$ is a scalar, the maximum absolute value over all elements of that output tensor. λ_{const} is a constant threshold. The functions min() and max() return the minimum and maximum elements, respectively. The term α represents the adaptive noise amplitude in the current epoch, while ϵ is sampled from a uniform distribution. In implementation, we do not store the full-student outputs from previous epochs on disk. Instead, $\max_{b,l,d}(|Y^{e-1}|)$ is computed online as a running maximum over mini-batches during epoch $e-1$, and only this scalar is retained for epoch e. Therefore, the additional memory overhead is negligible.

The supervision signal is derived from the consistency between the teacher and student networks' outputs, measured by their L1 distance, which is formulated as follows:

$$\mathcal{L}_{kd} = \frac{1}{b \times l \times d}\sum_{i=1}^{l}\sum_{j=1}^{b}\left|y_{ij} - y_{ij}^{\text{pred}}\right| \tag{5}$$

where $y_{ij} \in \mathbb{R}^d$ and $y_{ij}^{\text{pred}} \in \mathbb{R}^d$ denote the outputs of the teacher and student networks, respectively, for the ith token in the jth batch. The indices i and j are integers ranging from 1 to l and 1 to b, respectively.

It is important to note that although SA-1B [13] is a natural-image dataset, our goal is to distill SAM into a student model that transfers well to remote sensing imagery, where the data distribution differs substantially. Remote sensing images typically have higher spatial resolution, different sensor characteristics, and more complex high-frequency textures. To reduce this domain gap during distillation, we introduce an RS-motivated augmentation that injects high-frequency perturbations into SA-1B [13] images. Specifically, we add a mixture of noise and masks to emphasize high-frequency components with randomly sampled intensities. This approach encourages the student to learn robust, shape- and structure-oriented representations rather than overfitting to low-frequency appearance cues in natural images, thereby improving its adaptation to remote sensing imagery. Relevant experiments validate the reasonableness of this approach. We apply this perturbation only during distillation (pretraining), while keeping the downstream RS fine-tuning unchanged.

B. Efficient Remote Sensing Instance Segmentation

The efficient segmentation model consists of three primary components: backbone, neck, and head. The backbone architecture is derived from previously discussed knowledge distillation techniques. The neck component performs multilevel feature fusion, while the head component executes task-specific operations.

1) Multilevel Feature Fusion Neck: The neck serves as the connecting component between the backbone and head, extracting high-level semantic information through feature abstraction. As illustrated in Fig. 4, it comprises three key components: a downsampled feature layer, a cascade of residuals, and SimpleFPN. The downsampled feature layer reshapes

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

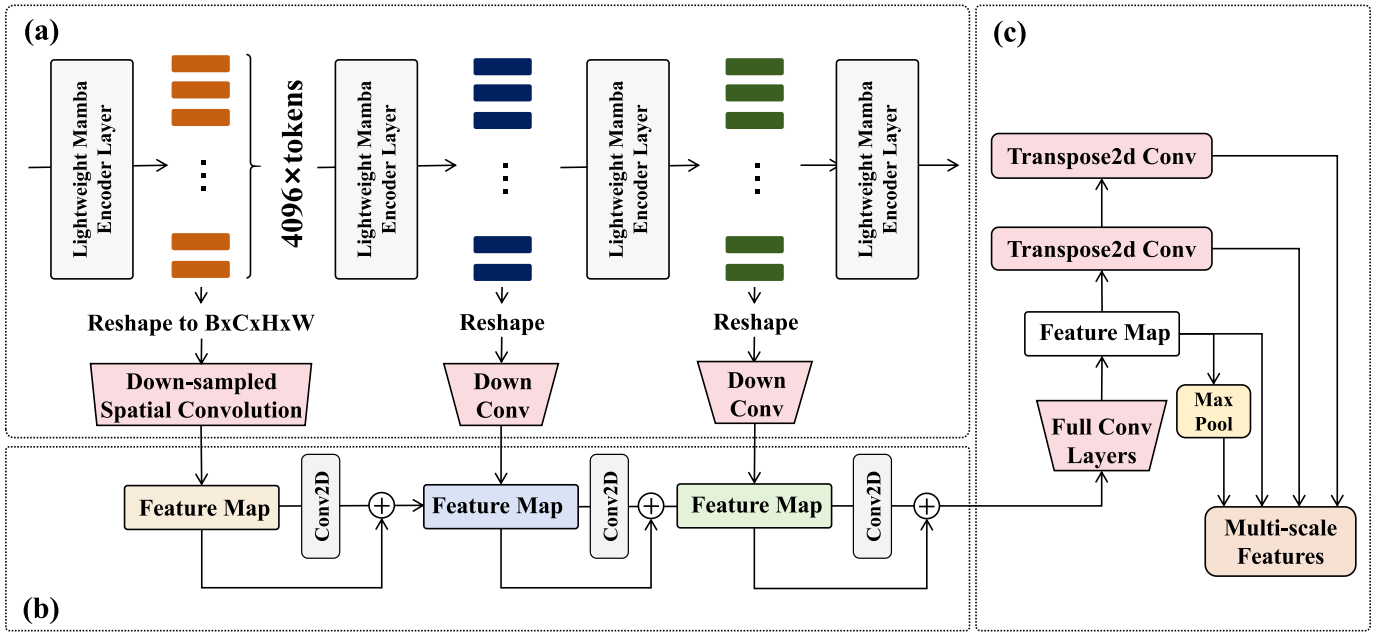


Fig. 4. Structure of neck part of the proposed model. The neck consists of three parts. (a) Multilevel token feature downsampling module. (b) Cascaded residual module. (c) Simple FPN module.

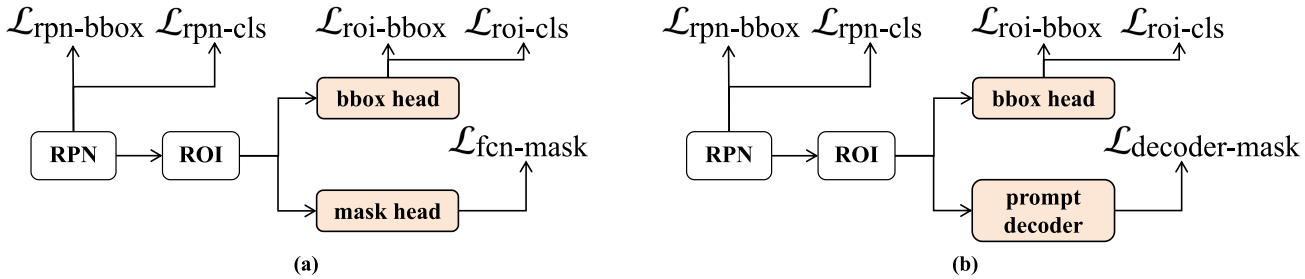


Fig. 5. Structure of the head part of the proposed model. We show two different ways of implementation of the network's head part for instance segmentation. (a) Box head. (b) Prompt head.

sequences from odd layers while performing spatial downsampling. To preserve shallow features for edge detail refinement, the cascade of residuals network implements skip connections between deep and shallow feature layers. The SimpleFPN generates standard multiscale feature maps through cascading downsampling and upsampling convolutions.

We fuse multidepth features and build multiscale maps at the final embedding stage. Unlike CNN backbones, SSM/ViT backbones typically operate on a single token/grid resolution and do not naturally produce an explicit multiresolution feature pyramid across stages. Constructing a progressive FPN pyramid by injecting features into multiple intermediate upsampling layers would require additional downsampling/upsampling operators and repeated fusion at several levels, increasing computation and undermining our goal of a lightweight, linear-time design. Instead, we aggregate complementary information from multiple depths at a unified resolution via lightweight skip connections, and then use a minimal SimpleFPN to output standard multiscale feature maps for the detector. This multilevel feature fusion neck operates on minimal spatial and channel dimensions, resulting in highly efficient computational costs.

Remote sensing images often contain sparse small objects, ranging from small vehicles to ships, and do not exhibit the abundance of large-scale instances found in natural images, such as those in the COCO dataset, where one object may cover a significant portion of the image. Since our backbone design incorporates dense patches, this allows the model to focus more on smaller objects. Therefore, when designing a neck that complements this backbone, we prefer to strategically select and fuse features from specific intermediate layers of the SSM backbone, rather than structuring the model like an ResNet + FPN to extract and merge multiscale features. This approach enhances the model's feature extraction capability in remote sensing images compared with natural images.

2) Instance Segmentation Head: As illustrated in Fig. 5, the segmentation head is designed with two variants: a box-based head and a prompt-based head. The box-based head follows the approach of Mask R-CNN [41], incorporating a region proposal network (RPN) head and a region of interest (RoI) head. The total loss function is formulated as follows:

$$\mathcal{L} = \mathcal{L}_{rpn-cls} + \mathcal{L}_{rpn-bbox} + \mathcal{L}_{roi-cls} + \mathcal{L}_{roi-bbox} + \mathcal{L}_{fcn-mask} \quad (6)$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

where $\mathcal{L}_{rpn-cls}$ and $\mathcal{L}_{rpn-bbox}$ denote the classification and bounding box losses of the RPN, respectively; $\mathcal{L}_{roi-cls}$ and $\mathcal{L}_{roi-bbox}$ represent the RoI classification and regression losses, respectively; and $\mathcal{L}_{fcn-mask}$ represents the binary cross-entropy mask loss computed by a simple fully convolutional network (FCN).

The prompt-based head eliminates the FCN component present in the box-based head and instead feeds the RoI features directly into the SAM prompt encoder and decoder to generate binary masks. The corresponding loss function is defined as follows:

$$\mathcal{L} = \mathcal{L}_{rpn-cls} + \mathcal{L}_{rpn-bbox} + \mathcal{L}_{roi-cls} + \mathcal{L}_{roi-bbox} + \mathcal{L}_{decoder-mask} \tag{7}$$

where $\mathcal{L}_{decoder-mask}$ represents the loss between the decoder-generated segmentation output and the ground-truth mask.

It should be noted that the box head adheres to a traditional Mask R-CNN design, where RoI-aligned features are input into an FCN mask predictor to produce the instance mask. In contrast, the Prompt Head replaces the FCN mask predictor with a procedure that employs SAM-style prompt encoding and mask decoding. In this approach, RoI-aligned features are first mapped into a small set of learned prompt tokens, which act as implicit prompts derived from the detected RoI region rather than from user interactions like clicks or bounding boxes. Thus, in our context, prompt specifically refers to the learned prompt tokens constructed from RoI features, rather than manually provided prompts.

C. Implementation Details

1) Network Design: RS4D is implemented using MMDetection [90] and MMEngine [91] frameworks, which organize the model architecture into three primary components: the backbone, the neck, and the head. Among the tested backbone variants, we focus on ScanningMamba due to its superior convergence rate and performance in our preliminary experiments. The neck architecture adopts the feature aggregator design from RSPrompter [14], with modifications only to the number of aggregator layers.

2) Training Details: Following the model's structural design, the training process consists of two main phases: knowledge distillation of the backbone and fine-tuning of the complete model for instance segmentation tasks.

For backbone distillation, unless otherwise specified, we utilize 0.1% of the unlabeled data from the readily available SA-1B dataset [13], which consists of 10k images (0.1% of SA-1B [13]). For the NWPU experiments, we use distilled weights obtained from 10% (1000k) of SA-1B to mitigate the data scarcity of downstream training. In distillation, we choose to use only a subset for distillation for two main reasons. First, it allows us to explore computational and storage efficiency while keeping the overall transfer process low-cost. Importantly, even this small subset is sufficient to convey meaningful segmentation priors from the SAM encoder to the lightweight SSM student, aligning with our goal of efficient knowledge transfer. The second reason is that the complete SA-1B dataset [13] contains over 11 million images, which incurs substantial training and storage costs, comparable to pretraining models using MAE, and contradicts our original intention. Note that 10k images refers to the number of unique SA-1B images in the subset. The SA-1B subsets are constructed directly following the official data organization and download order. During distillation, we apply stochastic augmentations multiple times, resulting in roughly 50k augmented training samples. We emphasize that the 0.1% setting should be viewed as an efficient default rather than a universal saturation point, and the optimal distillation data volume depends on the downstream dataset regime.

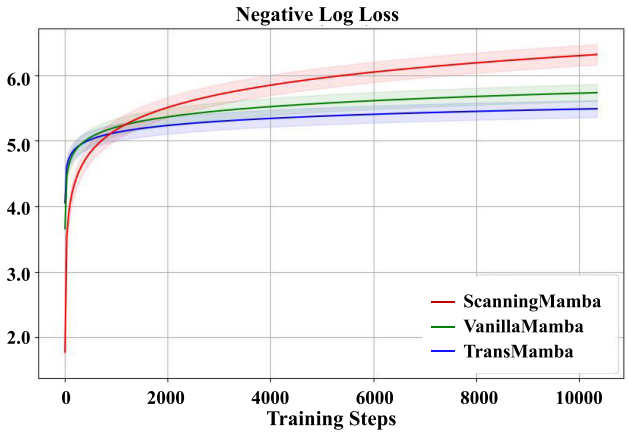


Fig. 6. Comparison of backbone architectures in terms of the negative log loss and training steps, where higher values indicate faster convergence. The shaded region around each curve represents the loss variance calculated using a sliding window, providing insight into training stability.

Although there is a domain gap between this dataset and remote sensing imagery, this discrepancy has a limited effect on downstream tasks, as we are mainly transferring general knowledge. All images are resized to 1024×1024 resolution and fed into the teacher model. Image flipping is also used as a data augmentation technique. Simultaneously, masked and noise-augmented versions of the same images are input to the SSM student model for feature extraction. L1 distance is applied to the final outputs to ensure that the deep features extracted by both models remain closely aligned. As demonstrated in Fig. 6, ScanningMamba achieves the optimal balance between performance and efficiency, along with more stable convergence. This indicates that bidirectional scanning enhances the completeness of features for dense mask prediction while maintaining the linear-time advantage of SSM. Consequently, we have selected ScanningMamba as the default backbone for the subsequent fine-tuning phase.

The instance segmentation model's backbone is initialized using the distilled SSM, and all model parameters are fine-tuned on the specific remote sensing instance segmentation dataset. The model is trained for 800 epochs. We implement basic data augmentation techniques, including random flipping, resizing, and cropping. For optimization, we employ the AdamW optimizer with an initial learning rate of 1×10^{-4} and a weight decay of 0.05. The learning rate is adjusted using a cosine annealing scheduler with linear warm-up. Training is conducted using bfloat16 precision. Other parameters use the default configuration of MMDetection [90], considering

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

the robustness of the framework, and for the sake of model reproducibility, while reducing training costs. We did not use multiple random seeds for averaging. In the distillation phase, we built the code using native PyTorch, with a random seed set to 42.

IV. Experiments and Analysis

A. Experimental Setup

1) Dataset and Evaluation Metrics: We fine-tune and evaluate RS4D on three publicly available remote sensing instance segmentation datasets: the SAR ship detection dataset (SSDD) [38], the Building Extraction Dataset (WHU) [39], and the NWPU VHR-10 Dataset (NWPU) [40]. All experimental settings are consistent with those used in RSPrompter.

SSDD [38] is a public SAR ship dataset that primarily contains small targets. The dataset consists of 500×500 pixel images with spatial resolutions ranging from 1 to 15 m. The targets are annotated using horizontal bounding boxes.

WHU [39] is a comprehensive aerial building dataset containing high-quality imagery. The dataset comprises 512×512 pixel images with a spatial resolution of 0.3 meters. It is divided into 4736 training images, 1036 validation images, and 2416 testing images.

NWPU [40] encompasses ten object categories: ships, aircraft, ports, bridges, oil tanks, baseball fields, tennis courts, basketball courts, track and field facilities, and cars. The dataset is relatively small, consisting of 650 target images and 150 background images.

Due to the relatively small size of the NWPU dataset, along with its significant variations in target scale and the number of categories, we initialize the backbone using distilled weights obtained from a larger subset of SA-1B (10%) and employed test time augmentation during testing.

To comprehensively evaluate the performance of the proposed method, we adopt two sets of metrics to assess the model's accuracy and efficiency. For accuracy, we use mean average precision (mAP), specifically AP_{bbox} for bounding boxes, AP_{mask} for instance masks, and AP^{50} and AP^{75} for IoU thresholds of 0.5 and 0.75, respectively. For efficiency, we measure the FLOPs and the number of parameters.

B. Qualitative Analysis

We compare the proposed RS4D with other state-of-the-art approaches, including traditional instance segmentation models, such as Mask R-CNN [41], Mask Scoring R-CNN [42], HTC [43], SOLOv2 [45], CondInst [44], BoxInst [92], Mask2Former [46], and Cascade Mask RCNN [93]. We also compared the typical models for natural images, including SwinV2 (HTC++) [94], CBNetv2 [95], and ViTDet [96]. In addition, we compare against specialized remote sensing models including CATNet [53] and HQ-ISNet [10], as well as foundation model variants [14]: SAM-cls, SAM-det, SAM-seg, RSPrompter (anchor), and RSPrompter (query). It is important to note that although SwinV2 (HTC++) [94] and ViTDet [96] are not pretrained foundation models like SAM, they also use ViT as their backbone, and have a comparable or similar number of parameters to the foundational model.

During training, the model's training configuration remains consistent, as detailed in Section III-C2. In the ablation experiments, all configurations except for the ablated variables are kept the same. The variables that are changed are the masking ratio and the noise ratio used in pretraining, both of which are selected from 0%, 10%, 20%, and 30%.

1) Comparative Analysis on the SSDD Dataset: The performance comparison between RS4D and other methods on the SSDD dataset is presented in Table I, with visual examples shown in Fig. 7. The shaded cells indicate the top two performing models, with darker colors representing higher rankings. RS4D outperforms both traditional approaches and recent foundation models, achieving 70.7 AP_{bbox} and 70.0 AP_{mask} with its box-based variant. The prompt-based RS4D demonstrates slightly lower performance than its box-based counterpart, yet still ranks fifth in detection metrics among the 22 evaluated methods. Furthermore, while maintaining a similar parameter count to CondInst and BoxInst, RS4D demonstrates significantly superior performance. Despite the substantial modality difference between SAR data and the SA-1B dataset [13], experimental results indicate that our method exhibits robust generalization capabilities and high efficiency even when transferred to a limited dataset.

2) Comparative Analysis on the WHU Dataset: The quantitative comparison results on the WHU dataset are presented in Table II, with visual examples shown in Fig. 8. While RS4D significantly outperforms traditional models, it performs marginally below certain foundation models. Specifically, RS4D (box) achieves scores of 70.3/70.1 (AP_{bbox}/AP_{mask}), which is comparable to the SAM-seg (Mask RCNN) foundation model (70.3/70.1), but falls short of the current state-of-the-art foundation model RSPrompter (query), which achieves 72.5/72.5.

3) Comparative Analysis on the NWPU Dataset: The experimental results and visualizations on the NWPU dataset are presented in Table III and Fig. 9, respectively. On this dataset, RS4D shows competitive performance with clear efficiency advantages, although its gains over existing methods are more modest than those on SSDD and WHU. Specifically, RS4D (box) achieves 61.7/60.8 (AP_{bbox}/AP_{mask}), while RS4D (prompt) achieves 61.5/60.3(AP_{bbox}/AP_{mask}). These results indicate that the proposed method remains competitive in mask prediction under a much lighter backbone, but it does not outperform the strongest baselines in (AP_{bbox}). This relatively moderate performance can be attributed to the HiPPO-based information compression in Mamba, which may overlook some small objects and densely arranged instances in NWPU, especially under the limited amount of downstream training data.

4) Model Efficiency Analysis: We first evaluate the efficiency of our backbone in terms of parameter count and computational complexity (FLOPs), comparing it with established architectures including ResNet18 [98], EfficientNet b0 [99], AlexNet [100], VGG16 [101], MobileNet v2 [102], DenseNet [103], GoogleNet [104], ConvNet-b [105], and ViT [74]. The comparative analysis is presented in Fig. 10 and Table IV. The backbone of RS4D demonstrates comparable

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON WITH OTHER METHODS ON THE SSDD DATASET [38]

Method	Backbone	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}	Params (M)	bb-Params (M)	bb-FLOPs (G)
Mask RCNN [41]	ResNet50	67.7	95.6	84.9	64.3	92.6	80.9	44.18	25.56	172.69
MS RCNN [42]	ResNet50	67.8	95.3	85.9	64.9	93.3	80.4	60.74	25.56	172.69
HTC [43]	ResNet50	69.3	97.1	85.7	64.1	94.4	80.6	46.59	25.56	172.69
SOLO v2 [45]	ResNet50	-	-	-	58.5	86.2	74.0	46.58	25.56	172.69
SCNet [97]	ResNet50	66.9	92.5	82.5	64.9	92.6	80.1	94.80	25.56	172.69
CondInst [44]	ResNet50	68.1	92.4	85.5	62.5	93.4	81.2	34.16	25.56	172.69
BoxInst [92]	ResNet50	62.8	96.2	74.7	45.2	92.3	35.3	35.14	25.56	172.69
Mask2Former [46]	ResNet50	62.7	90.7	75.6	64.4	93.0	82.4	44.02	25.56	172.69
CATNet [53]	ResNet50	67.5	96.8	80.4	63.9	93.7	80.1	54.77	25.56	172.69
HQ-ISNet [10]	ResNet50	66.6	95.9	80.2	63.4	95.7	78.1	80.56	25.56	172.69
Cascade Mask RCNN [93]	ResNet50	70.4	97.5	85.9	67.9	96.6	85.5	77.02	25.56	172.69
SwinV2 (HTC++) [94]	SwinV2-Huge	59.4	85.5	76.8	56.4	85.6	70.0	709.09	655.14	303.37
CBNetv2 [95]	Swin-Tiny	40.6	62.7	48.5	38.6	62.5	46.8	75.58	28.30	12.47
ViTDet [96]	Swin-Huge	62.4	95.1	73.0	61.4	94.9	77.2	679.42	655.58	375.10
SAM-seg (Mask RCNN) [14]	ViT-Base	68.7	97.2	84.3	66.1	94.5	83.7	309.69	89.67	972
SAM-seg (Mask2Former) [14]	ViT-Base	63.0	94.9	75.6	66.5	95.0	83.6	112.16	89.67	972
SAM-cls [14]	ViT-Base	43.2	70.8	48.8	47.5	78.1	57.7	-	-	-
SAM-det [14]	ViT-Base	70.0	95.8	85.3	46.0	93.8	37.0	135.08	89.67	972
RSPrompter [14](anchor)	ViT-Base	70.4	97.7	86.2	66.8	94.7	84.0	117.09	89.67	972
RSPrompter [14](query)	ViT-Base	66.0	95.6	78.7	67.3	95.6	84.3	101.02	89.67	972
RS4D (box)	SSM	70.7	97.4	86.4	70.0	97.4	87.5	33.20	11.75	108.8
RS4D (prompt)	SSM	69.7	96.6	83.2	64.7	93.7	80.9	38.83	11.75	108.8

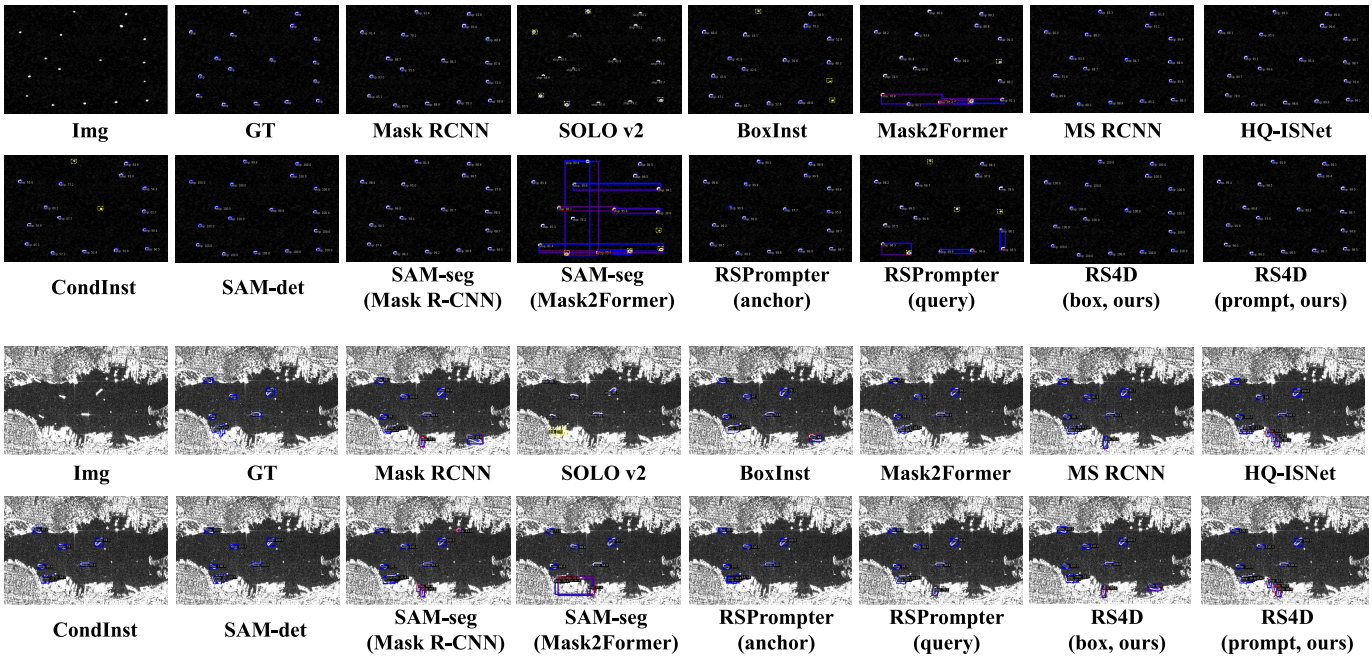


Fig. 7. Images between the proposed method and some SOTA instance segmentation methods on the infrared ship instance segmentation dataset SSDD [38]. Red bounding boxes represent false positives, while yellow bounding boxes indicate false negatives.

parameter counts and FLOPs to ultralightweight CNN models such as MobileNetv2 and EfficientNet b0. Notably, compared to ViT, our model achieves an $\times 8$ reduction in parameter count and a $\times 9$ reduction in FLOPs, making it particularly suitable for real-time applications and resource-constrained environments.

Furthermore, we conducted a comprehensive comparison of model efficiency and instance segmentation performance between RS4D and other instance segmentation models. As shown in Table V and Fig. 1, on the SSDD dataset, RS4D achieves comparable or slightly superior metrics to the large model RSPrompter while requiring only 1/4 of the parameters and 1/9 of the FLOPs. In addition, when compared with models with similar parameter counts such as CondInst and BoxInst, RS4D demonstrates superior performance.

C. Ablation Study

In this section, we conduct comprehensive ablation studies to validate the effectiveness of our proposed method and its components.

1) Effects of Knowledge Distillation: To investigate the effect of model initialization, we conducted comparative experiments using distilled weights versus random initialization for

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

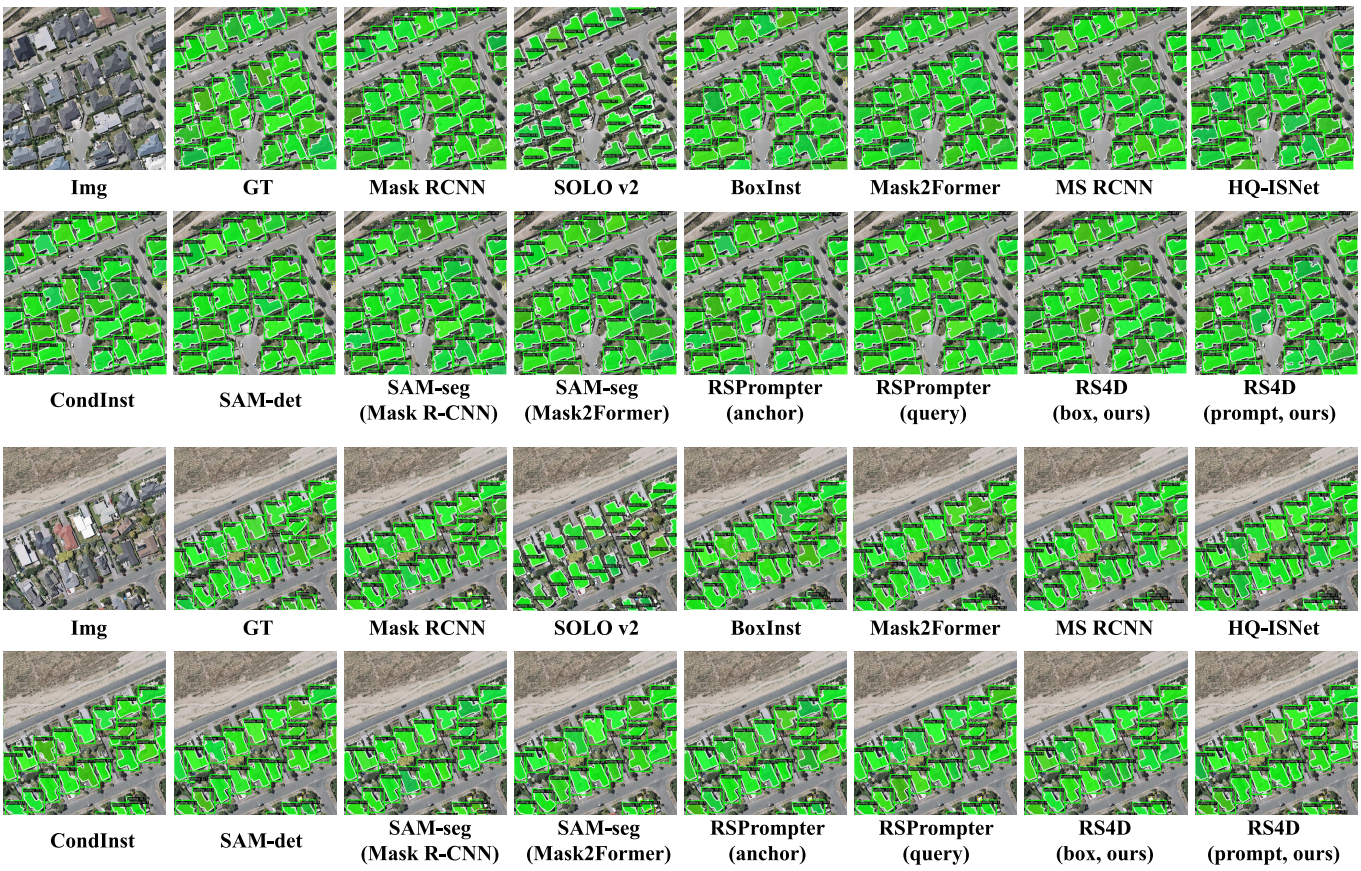


Fig. 8. Images between the proposed method and some SOTA instance segmentation methods on the building instance segmentation dataset WHU [39]. Red bounding boxes represent false positives, while yellow bounding boxes indicate false negatives.

TABLE II
COMPARISON WITH OTHER METHODS ON THE WHU DATASET [39]

Method	Backbone	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}	Params (M)	bb-Params (M)	bb-FLOPs (G)
Mask RCNN [41]	ResNet50	66.4	86.3	76.3	65.6	87.1	76.7	44.18	25.56	172.69
MS RCNN [42]	ResNet50	67.7	87.2	77.1	66.9	87.5	77.5	60.74	25.56	172.69
HTC [43]	ResNet50	68.4	87.8	77.8	67.7	88.1	78.3	46.59	25.56	172.69
SOLO v2 [45]	ResNet50	-	-	-	65.2	86.7	75.2	46.58	25.56	172.69
SCNet [97]	ResNet50	68.1	87.6	77.7	66.5	87.9	81.2	94.80	25.56	172.69
CondInst [44]	ResNet50	66.7	87.5	76.7	66.6	87.8	78.7	34.16	25.56	172.69
BoxInst [92]	ResNet50	66.7	86.4	75.7	55.0	86.5	63.2	35.14	25.56	172.69
Mask2Former [46]	ResNet50	69.3	90.7	75.6	64.4	93.0	82.4	44.02	25.56	172.69
CATNet [53]	ResNet50	66.7	86.3	76.4	66.1	86.6	76.8	54.77	25.56	172.69
HQ-ISNet [10]	ResNet50	66.1	86.0	75.7	66.5	86.4	78.9	80.56	25.56	172.69
Cascade Mask RCNN [93]	ResNet50	65.3	83.6	75.1	64.5	84.5	75.4	77.02	25.56	172.69
SwinV2 (HTC++) [94]	SwinV2-Huge	46.8	62.2	54.5	46.7	62.5	56.2	709.09	655.14	303.37
CBNetv2 [95]	Swin-Tiny	60.0	86.6	73.6	58.8	85.8	74.6	75.58	28.30	12.47
ViTDet [96]	Swin-Huge	61.1	82.9	71.2	59.3	83.1	69.9	679.42	655.58	375.10
SAM-seg (Mask RCNN) [14]	ViT-Base	70.3	89.8	81.9	70.1	89.9	81.0	309.69	89.67	972
SAM-seg (Mask2Former) [14]	ViT-Base	70.7	88.4	79.1	71.1	89.5	81.1	112.16	89.67	972
SAM-cls [14]	ViT-Base	46.8	65.7	53.5	49.3	71.2	57.6	-	-	
SAM-det [14]	ViT-Base	69.1	90.1	79.2	61.8	89.1	71.0	135.08	89.67	972
RSPrompter (anchor) [14]	ViT-Base	71.9	90.0	82.8	70.4	90.0	80.5	117.09	89.67	972
RSPrompter (query) [14]	ViT-Base	72.5	91.0	81.7	72.5	92.0	82.9	101.02	89.67	972
RS4D (box)	SSM	70.3	89.3	79.2	70.1	89.5	80.3	33.20	11.75	108.8
RS4D (prompt)	SSM	67.7	89.0	77.6	64.4	88.3	75.0	38.83	11.75	108.8

the backbone. The experiments were performed on the NWPU dataset, with results presented in Fig. 11 and Table VI. The results demonstrate that initializing network parameters with distilled weights achieved superior performance compared with random initialization without the foundational model prior knowledge. As shown in Fig. 11, the model with distilled weight initialization also exhibited better convergence characteristics.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

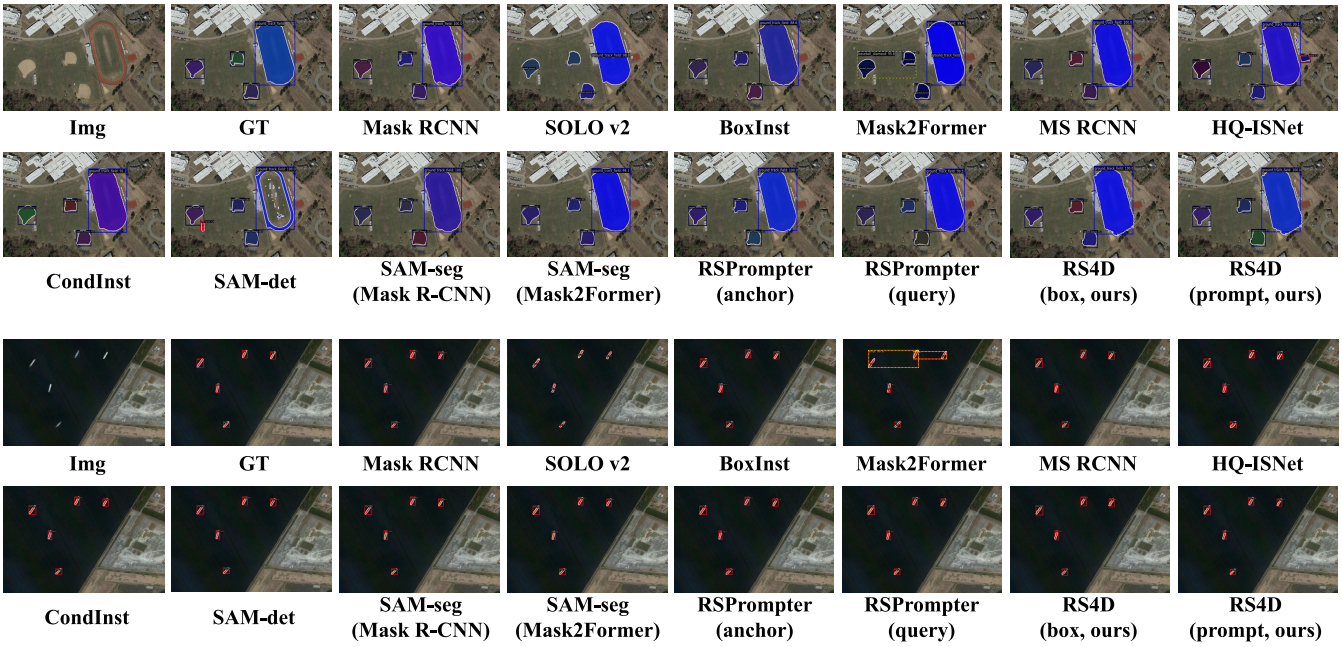


Fig. 9. Images between the proposed method and some SOTA instance segmentation methods on the instance segmentation dataset NWPU [40]. Red bounding boxes represent false positives, while yellow bounding boxes indicate false negatives.

TABLE III
COMPARISON WITH OTHER METHODS ON THE NWPU DATASET [40]

Method	Backbone	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}	Params (M)	bb-Params (M)	bb-FLOPs (G)
Mask RCNN [41]	ResNet50	62.3	88.3	75.2	59.7	89.2	65.6	44.18	25.56	172.69
MS RCNN [42]	ResNet50	62.3	88.6	73.1	60.7	88.7	67.7	60.74	25.56	172.69
HTC [43]	ResNet50	63.9	88.9	75.4	60.9	88.6	64.4	46.59	25.56	172.69
SOLO v2 [45]	ResNet50	-	-	-	50.9	77.5	54.1	46.58	25.56	172.69
SCNet [97]	ResNet50	60.0	87.5	69.1	58.1	87.4	62.0	94.80	25.56	172.69
CondInst [44]	ResNet50	62.3	87.8	73.3	59.0	88.5	62.8	34.16	25.56	172.69
BoxInst [92]	ResNet50	64.8	89.3	73.0	47.6	77.2	51.3	35.14	25.56	172.69
Mask2Former [46]	ResNet50	57.4	75.5	63.7	58.8	83.1	63.5	44.02	25.56	172.69
CATNet [53]	ResNet50	63.2	89.0	73.8	60.4	89.6	65.5	54.77	25.56	172.69
HQ-ISNet [10]	ResNet50	63.5	89.9	75.0	60.4	89.6	64.1	80.56	25.56	172.69
Cascade Mask RCNN [93]	ResNet50	67.5	92.4	77.3	63.5	92.7	66.9	77.02	25.56	172.69
SwinV2(HTC++) [94]	SwinV2-Huge	43.5	79.5	43.4	39.7	66.6	40.4	709.09	655.14	303.37
CBNetv2 [95]	Swin-Tiny	41.0	70.3	42.4	40.8	68.8	41.5	75.58	28.30	12.47
ViTDet [96]	Swin-Huge	54.8	85.4	64.7	49.6	80.4	56.6	679.42	655.58	375.10
SAM-seg (Mask RCNN) [14]	ViT-Base	68.8	92.2	80.1	65.2	92.0	71.6	309.69	89.67	972
SAM-seg (Mask2Former) [14]	ViT-Base	63.1	86.3	70.6	64.3	89.6	70.1	112.16	89.67	972
SAM-cls [14]	ViT-Base	40.2	57.1	44.5	44.0	66.0	47.6	-	-	-
SAM-det [14]	ViT-Base	64.2	89.6	74.6	51.5	74.8	54.0	135.08	89.67	972
RSPrompter (anchor) [14]	ViT-Base	70.3	93.6	81.0	66.1	92.7	70.6	117.09	89.67	972
RSPrompter (query) [14]	ViT-Base	68.4	90.3	74.0	67.5	91.7	74.8	101.02	89.67	972
RS4D (box)	SSM	61.7	85.8	70.1	60.8	90.7	64.4	33.20	11.75	108.8
RS4D (prompt)	SSM	61.5	86.9	73.7	60.3	87.4	63.3	38.83	11.75	108.8

2) Effects of Feature Quantity on Neck Fusion: This section investigates the impact of neck feature composition on instance segmentation performance. We constructed the neck component by selecting features from different hidden layers of the SSM backbone. The experimental results comparing different layer combinations are presented in Tables VII–IX. They were conducted on the SSDD, WHU, and NWPU datasets, respectively. The results demonstrate that appropriately increasing the number of selected hidden layers facilitates the fusion of deep and shallow features, thereby enhancing semantic segmentation representation. When using only a single layer (i.e., the final layer features), the model exhibited the poorest performance.

3) Effects of Noising and Masking in Distillation: Table X demonstrates the impact of varying noise levels and masking proportions against the accuracy using the SSDD dataset. Tables XI and XII were conducted on the WHU and NWPU datasets, respectively. The analysis reveals that controlled noise injection enhances object detection capabilities, while systematic masking strategies improve segmentation accuracy. Optimal performance was achieved with a 20% perturbation ratio; higher ratios led to performance degradation. The strategic application of noise and masking techniques during training resulted in enhanced model robustness and improved overall performance metrics.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

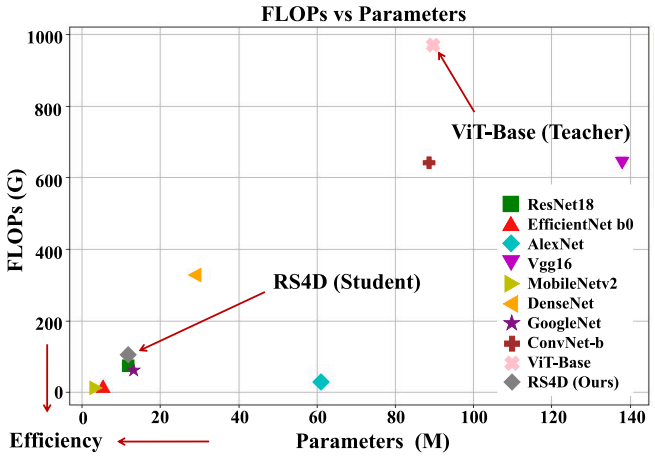


Fig. 10. Comprehensive comparison of computational efficiency across different models.

TABLE IV
COMPARISON OF RS4D'S BACKBONE WITH EXISTING ADVANCED BACKBONE ARCHITECTURES IN TERMS OF FLOPS AND PARAMETER COUNTS

Method	Parameters (M)	FLOPs (G)
ResNet18 [98]	11.69	76.32
EfficientNet b0 [99]	5.29	17.56
AlexNet [100]	61.10	30.02
Vgg16 [101]	138.00	642.00
MobileNet v2 [102]	3.51	13.92
DenseNet [103]	28.68	330.00
GoogleNet [104]	13.01	63.28
ConvNet-b [105]	88.59	642.51
ViT-Base [74]	89.67	972.00
RS4D (Ours)	11.75	108.8

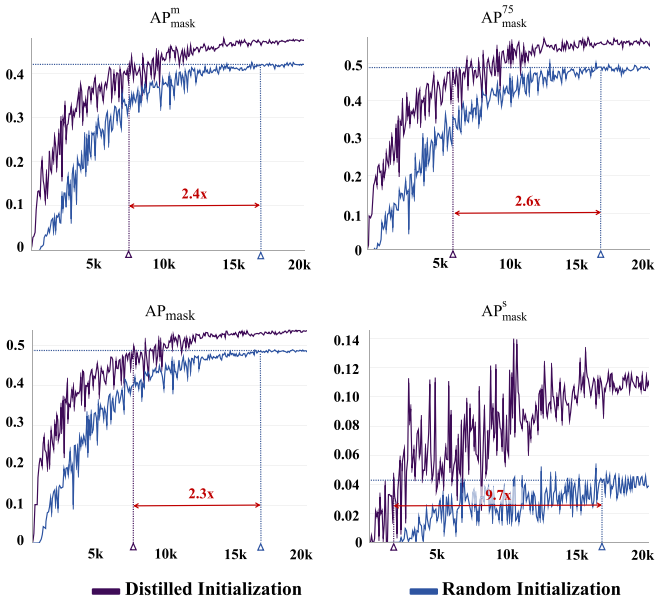


Fig. 11. Performance across training steps for backbone initialization with and without distilled weights on the NWPU dataset.

4) Effects of Backbone Depth and Width: Our parametric study using the SSDD dataset examined how varying the depth (number of layers) and width (dimension) of the SSM backbone affected model performance. As illustrated in Tables XIII–XV, modest increases in both parameters yielded improved overall performance. However, arbitrarily increasing model parameters without considering computational resources and dataset constraints proves counterproductive, as it substantially elevates the risk of overfitting.

5) Effects of Distillation Data Volume: Our experimental study examined the impact of distillation data volume on model performance, starting with only 0.1% of the SA-1B dataset [13]. We gradually increased the data volume to 10k (0.1%), 25k (0.25%), 100k (1%), and 1000k (10%) samples. As shown in Table XVI, increasing the distillation data volume consistently improves downstream RS instance segmentation performance, especially AP_{mask}, indicating that the proposed distillation procedure can benefit from larger scale unlabeled data when computational resources permit. This suggests that a larger dataset provides richer information, enabling better generalization. Conversely, the suboptimal performance observed in the NWPU dataset highlights the importance of sufficient training data for effective knowledge transfer. These results suggest that the required distillation volume depends on the downstream data regime. Smaller and more category-diverse datasets, such as NWPU, can benefit more from larger distilled subsets, whereas a small subset can already provide an effective and efficient initialization for datasets such as SSDD and WHU. Therefore, in this work, the choice of distilled data volume should be understood as an empirical efficiency-performance tradeoff rather than a claim of a universal saturation threshold.

6) Effects of Frozen-Backbone: To assess representation quality, we additionally evaluate different backbone settings. Among them, "all-params" and "frozen-backbone" use the pretrained parameters we distilled, with the former being fully fine-tuned and the latter only fine-tuning the neck and head. "Zero-init" randomly initializes the parameters and performs full parameter fine-tuning. "Imagenet-init" replaces the backbone with the ImageNet pretrained EfficientNet-B1 and also performs full parameter fine-tuning. The frozen protocol still achieves strong performance, as seen in the added row in Table XVII, indicating that the distilled backbone learns transferable representations rather than relying solely on end-to-end adaptation. At the same time, we also compared RS4D with EfficientNet-B1, which has a similar number of backbone parameters and is pretrained on ImageNet. In the scenario of full-parameter fine-tuning, RS4D demonstrates a significant advantage.

D. Discussion and Limitation

As a lightweight instance segmentation architecture, RS4D shows considerable promise for model compression and edge computing applications. Through the transformation of sparse attention space into dense state space, the model effectively distills essential information while achieving significant compression in the state-space domain. This approach offers a particularly promising solution for foundation models with billions of parameters, potentially enabling efficient knowledge storage and rapid inference capabilities.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

TABLE V
MODEL EFFICIENCY AND PERFORMANCE COMPARISONS ON THE SSDD DATASET [38]

Metrics	Mask RCNN [41]	MS RCNN [42]	HTC [43]	SCNet [97]	CondInst [44]	BoxInst [92]	RSPrompter [14]	RS4D (Ours)
Parameters (M)	44.18	60.74	46.59	94.80	34.16	35.14	117.09	33.20
AP_{bbox}	67.7	67.8	69.3	66.9	68.1	62.8	70.4	70.7
AP_{mask}	64.3	64.9	64.1	64.9	62.5	45.2	66.8	70.0

TABLE VI
EFFECT OF BACKBONE INITIALIZATION WITH AND WITHOUT DISTILLED WEIGHTS ON THE NWPU DATASET [40]

Dist. Init.	AP_{mask}	AP^{75}_{mask}	AP^{m}_{mask}	AP^{s}_{mask}
w/	52.2	54.5	46.5	9.9
w/o	48.3	49.0	41.6	5.3

TABLE VII
PERFORMANCE OF NECK FUSION'S DIFFERENT MULTILAYER FEATURE SELECTION ON THE SSDD DATASET [38]

Layers	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}
24	70.5	97.5	85.2	68.7	97.4	87.2
12	70.7	97.4	86.4	70.0	97.4	87.5
6	69.3	96.7	84.4	69.3	95.8	86.7
4	71.0	97.7	86.5	70.0	97.6	86.8
3	69.3	97.6	84.1	69.6	97.6	86.3
2	69.6	97.7	85.5	69.0	96.7	85.6
1	1.70	0.67	0.01	0.06	0.28	0.01

TABLE VIII
PERFORMANCE OF NECK FUSION'S DIFFERENT MULTILAYER FEATURE SELECTION ON THE WHU DATASET [39]

Layers	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}
24	68.4	89.6	77.7	68.1	89.8	78.8
12	70.3	89.3	79.2	70.1	89.5	80.3
6	68.8	89.8	78.7	69.0	89.9	79.8
4	69.1	89.9	78.9	69.4	90.1	80.0
3	68.7	89.7	78.6	68.9	89.9	79.8
2	68.6	89.7	78.5	69.3	89.8	79.9
1	6.3	21.7	1.0	1.8	10.4	0.1

TABLE IX
PERFORMANCE OF NECK FUSION'S DIFFERENT MULTILAYER FEATURE SELECTION ON THE NWPU [40]

Layers	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}
24	61.2	86.6	72.3	59.0	85.6	63.4
12	60.6	85.0	69.3	57.7	83.6	62.1
6	58.4	82.6	67.0	56.1	82.5	59.8
4	60.8	87.8	69.3	58.2	86.7	62.4
3	59.9	84.8	70.7	57.1	84.2	61.6
2	59.7	85.3	69.0	57.7	83.6	60.7
1	3.8	10.7	1.9	2.2	7.5	0.5

TABLE X
INSTANCE SEGMENTATION PERFORMANCE OF VARYING NOISE AND MASKING RATIOS IN KNOWLEDGE DISTILLATION ON THE SSDD DATASET [38]

Noise	Mask	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}
0%	0%	70.4	96.8	86.2	68.7	95.7	85.9
10%	0%	70.8	98.4	87.4	69.0	97.5	84.7
0%	10%	70.2	97.7	85.3	69.3	97.7	87.5
10%	10%	69.2	96.6	83.6	69.3	97.4	86.7
20%	20%	70.7	97.4	86.4	70.0	97.4	87.5
30%	30%	69.2	97.6	83.5	69.2	96.8	86.7

TABLE XI
INSTANCE SEGMENTATION PERFORMANCE OF VARYING NOISE AND MASKING RATIOS IN KNOWLEDGE DISTILLATION ON THE WHU DATASET [39]

Noise	Mask	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}
0%	0%	69.0	88.4	78.7	69.4	89.3	80.0
10%	0%	69.1	89.8	78.7	68.5	89.9	79.6
0%	10%	68.8	89.2	78.8	69.0	89.4	80.1
10%	10%	69.0	89.7	78.7	69.2	89.9	79.8
20%	20%	70.3	89.3	79.2	70.1	89.5	80.3
30%	30%	68.0	89.2	77.9	68.2	89.3	79.1

TABLE XII
INSTANCE SEGMENTATION PERFORMANCE OF VARYING NOISE AND MASKING RATIOS IN KNOWLEDGE DISTILLATION ON THE NWPU DATASET [40]

Noise	Mask	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}
0%	0%	57.0	83.8	66.2	54.7	81.6	57.8
10%	0%	56.2	80.2	65.3	54.3	79.2	57.4
0%	10%	58.6	83.4	67.8	56.4	82.6	60.4
10%	10%	59.2	84.6	67.7	56.5	84.0	60.0
20%	20%	60.6	85.0	69.3	57.7	83.6	62.1
30%	30%	57.6	81.9	64.6	54.8	81.8	57.1

TABLE XIII
INSTANCE SEGMENTATION PERFORMANCE OF SSM BACKBONE'S DEPTH AND WIDTH ON THE SSDD DATASET [38]

Depth	Width	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}
16	256	68.1	97.3	81.7	68.7	97.4	84.4
24	256	70.7	97.4	86.4	70.0	97.4	87.5
24	320	70.5	98.5	86.9	69.8	98.5	86.9
28	256	69.7	96.6	85.1	69.0	96.6	87.1
48	256	69.8	97.5	85.5	68.2	97.5	85.7
48	320	69.4	97.4	84.5	69.8	97.5	87.0

RS4D demonstrates superior performance compared with both conventional approaches and foundation models when tested on WHU and SSDD datasets. However, its performance on the NWPU dataset only exceeds that of some traditional models. This limitation can be attributed to the visual information compression process utilizing Hippo matrices [33], where small objects that occupy minimal image patches are frequently lost during information compression. While the target size in SSDD is similarly small, the issue is not as significant. However, we observe that this problem becomes more pronounced when the number of training images is limited, as

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

TABLE XIV

INSTANCE SEGMENTATION PERFORMANCE OF SSM BACKBONE'S DEPTH AND WIDTH ON THE WHU DATASET [39]

Depth	Width	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}
16	256	57.9	84.0	66.2	56.5	84.2	65.4
24	256	70.3	89.3	79.2	70.1	89.5	80.3
24	320	69.1	88.3	78.7	68.9	89.3	79.2
28	256	68.3	88.3	77.8	68.3	88.5	79.0
48	256	66.6	88.8	76.6	65.2	89.0	76.6

TABLE XV

INSTANCE SEGMENTATION PERFORMANCE OF SSM BACKBONE'S DEPTH AND WIDTH ON THE NWPU DATASET [40]

Depth	Width	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}
16	256	52.1	7.0	58.2	49.2	74.6	51.7
24	256	60.6	85.0	69.3	57.7	83.6	62.1
24	320	58.9	81.8	68.1	58.0	82.0	63.0
28	256	56.2	83.5	65.6	53.9	82.2	58.6
48	256	52.9	78.8	59.1	51.2	77.3	53.1

TABLE XVI

IMPACT OF DIFFERENT DATA VOLUME IN DISTILLATION ON THE NWPU DATASET [40]

Data	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}
10k (0.1%)	56.9	82.4	65.6	54.8	81.5	55.5
25k (0.25%)	57.9	81.6	67.4	55.1	80.7	57.2
100k (1%)	60.6	85.0	69.3	57.7	83.6	62.1
1000k (10%)	61.7	85.8	70.1	60.8	90.7	64.4

TABLE XVII

IMPACT OF FROZEN-BACKBONE IN DISTILLATION ON THE SSDD DATASET [38]

Setting	AP_{bbox}	AP^{50}_{bbox}	AP^{75}_{bbox}	AP_{mask}	AP^{50}_{mask}	AP^{75}_{mask}
all-params	70.7	97.4	86.4	70.0	97.4	87.5
frozen-backbone	68.9	96.6	84.4	68.6	96.6	85.7
Zero-init	66.3	96.2	82.3	66.0	96.7	82.0
imagenet-init	68.6	98.2	83.6	69.4	97.8	86.1

seen in the NWPU dataset. In addition, the NWPU dataset contains a dense arrangement of numerous objects within the same category, making it easy for many adjacent and semantically similar patches to be overlooked during HiPPO's information compression. Future enhancements could address this limitation through increased embedding dimensionality, improved local feature integration, and optimized selective mechanism design.

V. CONCLUSION

This article introduces RS4D, a novel framework for instance segmentation of remote sensing images that leverages SSMs with linear computational complexity. The framework effectively condenses information from extensive self-attention spaces into compact linear state spaces. Our investigation encompasses three distinct SSM backbone architectures and two implementations for instance segmentation heads. Comprehensive experimental results demonstrate RS4D's exceptional performance in remote sensing image instance segmentation, achieving a $\times 9$ reduction in FLOPs compared with ViT-based methods and a $\times 6$ reduction relative to ConvNet approaches, while showing especially strong performance on SSDD and WHU and competitive efficiency results on NWPU. Notably, RS4D achieves performance on par with the foundation model-based methods while utilizing only 1/4 of the model parameters, making it an efficient solution for deploying lightweight foundation models.

REFERENCES

[1] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, "Object detection in 20 years: A survey," *Proc. IEEE*, vol. 111, no. 3, pp. 257–276, Mar. 2023.

[2] G. Cheng and J. Han, "A survey on object detection in optical remote sensing images," *ISPRS J. Photogramm. Remote Sens.*, vol. 117, pp. 11–28, Jul. 2016.

[3] B. Chen, L. Liu, Z. Zou, and Z. Shi, "Target detection in hyperspectral remote sensing image: Current status and challenges," *Remote Sens.*, vol. 15, no. 13, p. 3223, Jun. 2023.

[4] Z. Zheng, Y. Zhong, J. Wang, A. Ma, and L. Zhang, "FarSeg++: Foreground-aware relation network for geospatial object segmentation in high spatial resolution remote sensing imagery," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 11, pp. 13715–13729, Nov. 2023.

[5] D. Zhao, B. Yuan, and Z. Shi, "Inherit with distillation and evolve with contrast: Exploring class incremental semantic segmentation without exemplar memory," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 10, pp. 11932–11947, Oct. 2023.

[6] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, "YOLACT: Real-time instance segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 9157–9166.

[7] X. Wang, T. Kong, C. Shen, Y. Jiang, and L. Li, "SOLO: Segmenting objects by locations," in *Proc. 16th Eur. Conf. Comput. Vis.-ECCV*, 2020, pp. 649–665.

[8] E. Xie et al., "PolarMask: Single shot instance segmentation with polar representation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 12190–12199.

[9] K. Sofiiuk, O. Barinova, and A. Konushin, "AdaptIS: Adaptive instance selection network," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 7355–7363.

[10] H. Su et al., "HQ-ISNet: High-quality instance segmentation for remote sensing imagery," *Remote Sens.*, vol. 12, no. 6, p. 989, Mar. 2020.

[11] J. Yuan, "Learning building extraction in aerial scenes with convolutional networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 11, pp. 2793–2798, Nov. 2018.

[12] L. Huan, N. Xue, X. Zheng, W. He, J. Gong, and G.-S. Xia, "Unmixing convolutional features for crisp edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 10, pp. 6602–6609, Oct. 2022.

[13] A. Kirillov et al., "Segment anything," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2023, pp. 4015–4026.

[14] K. Chen et al., "RSPrompter: Learning to prompt for remote sensing instance segmentation based on visual foundation model," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, pp. 1–17, 2024.

[15] D. Wang et al., "SAMRS: Scaling-up remote sensing segmentation dataset with segment anything model," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–10.

[16] S. Zagoruyko and N. Komodakis, "Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer," 2016, *arXiv:1612.03928*.

[17] K. Chen, W. Li, J. Chen, Z. Zou, and Z. Shi, "Resolution-agnostic remote sensing scene classification with implicit neural representations," *IEEE Geosci. Remote Sens. Lett.*, vol. 20, pp. 1–5, 2023.

[18] D. Hong et al., "SpectralGPT: Spectral remote sensing foundation model," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 8, pp. 5227–5244, Aug. 2024.

[19] B. Zhao, P. Han, and X. Li, "Vehicle perception from satellite," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 4, pp. 2545–2554, Apr. 2024.

[20] K. Chen, C. Liu, B. Chen, J. Zhang, Z. Zou, and Z. Shi, "RSRefSeg 2: Decoupling referring remote sensing image segmentation with foundation models," *IEEE Trans. Geosci. Remote Sens.*, vol. 64, pp. 1–20, 2026.

[21] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 16344–16359.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

[22] H. Zhang et al., "Poolingformer: Long document modeling with pooling attention," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12437–12446.

[23] C. Wu, F. Wu, T. Qi, Y. Huang, and X. Xie, "Fastformer: Additive attention can be all you need," 2021, *arXiv:2108.09084*.

[24] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," 2020, *arXiv:2004.05150*.

[25] K. Chen, Z. Zou, and Z. Shi, "Building extraction from remote sensing images with sparse token transformers," *Remote Sens.*, vol. 13, no. 21, p. 4441, Nov. 2021.

[26] W. Li et al., "Box2Mask: Box-supervised instance segmentation via level-set evolution," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 7, pp. 5157–5173, Jul. 2024.

[27] X. Wu, Z.-H. Cao, T.-Z. Huang, L.-J. Deng, J. Chanussot, and G. Vivone, "Fully-connected transformer for multi-source image fusion," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 3, pp. 2071–2088, Mar. 2025.

[28] W. Sun and Z. Chen, "Learning many-to-many mapping for unpaired real-world image super-resolution and downscaling," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 9874–9889, Dec. 2024.

[29] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," 2021, *arXiv:2111.00396*.

[30] A. Gu, K. Goel, A. Gupta, and C. Ré, "On the parameterization and initialization of diagonal state space models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 35971–35983.

[31] L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang, "Vision mamba: Efficient visual representation learning with bidirectional state space model," 2024, *arXiv:2401.09417*.

[32] A. Gu et al., "Combining recurrent, convolutional, and continuous-time models with linear state space layers," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 572–585.

[33] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré, "HiPPO: Recurrent memory with optimal polynomial projections," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1474–1487.

[34] Z. Wang, C. Li, H. Xu, X. Zhu, X. Huang, and H. Li, "MambaInst: Lightweight state space model for real-time instance segmentation," in *Proc. ICASSP - IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2025, pp. 1–5.

[35] C. Xiao, M. Li, Z. Zhang, D. Meng, and L. Zhang, "Spatial-mamba: Effective visual state space models via structure-aware state fusion," 2024, *arXiv:2410.15091*.

[36] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[37] D. Guo et al., "DeepSeek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025, *arXiv:2501.12948*.

[38] T. Zhang et al., "SAR ship detection dataset (SSDD): Official release and comprehensive data analysis," *Remote Sens.*, vol. 13, no. 18, p. 3690, Sep. 2021.

[39] S. Ji, S. Wei, and M. Lu, "Fully convolutional networks for multisource building extraction from an open aerial and satellite imagery data set," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 1, pp. 574–586, Jan. 2019.

[40] G. Cheng, J. Han, P. Zhou, and L. Guo, "Multi-class geospatial object detection and geographic image classification based on collection of part detectors," *ISPRS J. Photogramm. Remote Sens.*, vol. 98, pp. 119–132, Dec. 2014.

[41] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2961–2969.

[42] Z. Huang, L. Huang, Y. Gong, C. Huang, and X. Wang, "Mask scoring R-CNN," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 6409–6418.

[43] K. Chen et al., "Hybrid task cascade for instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 4974–4983.

[44] Z. Tian, C. Shen, and H. Chen, "Conditional convolutions for instance segmentation," in *Proc. Eur. Conf. Comput. Vis.*, Aug. 2020, pp. 282–298.

[45] X. Wang, R. Zhang, T. Kong, L. Li, and C. Shen, "SOLOv2: Dynamic and fast instance segmentation," in *Proc. Int. Conf. Adv. Neural Inf. Process. Syst.*, 2020, pp. 17721–17732.

[46] B. Cheng, I. Misra, A. G. Schwing, A. Kirillov, and R. Girdhar, "Masked-attention mask transformer for universal image segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 1290–1299.

[47] Y. Xiong et al., "EfficientSAM: Leveraged masked image pretraining for efficient segment anything," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 16111–16121.

[48] Z. Zhang, Y. Ma, E. Zhang, and X. Bai, "PSALM: Pixelwise segmentation with large multi-modal model," in *Proc. Eur. Conf. Comput. Vis.*, 2025, pp. 74–91.

[49] Y. Ji et al., "DDP: Diffusion model for dense visual prediction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 21684–21695.

[50] J. Chen, J. Lu, X. Zhu, and L. Zhang, "Generative semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2023, pp. 7111–7120.

[51] C. Zhou, C. C. Loy, and B. Dai, "Extract free dense labels from clip," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 696–712.

[52] T. Zhou et al., "Image segmentation in foundation model era: A survey," 2024, *arXiv:2408.12957*.

[53] Y. Liu, H. Li, C. Hu, S. Luo, Y. Luo, and C. W. Chen, "Learning to aggregate multi-scale context for instance segmentation in remote sensing images," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 1, pp. 595–609, Jan. 2025.

[54] T. Shaharabany, A. Dahan, R. Giryes, and L. Wolf, "AutoSAM: Adapting SAM to medical images by overloading the prompt encoder," 2023, *arXiv:2306.06370*.

[55] C. Zhang, D. Han, S. Zheng, J. Choi, T.-H. Kim, and C. S. Hong, "MobileSAMv2: Faster segment anything to everything," 2023, *arXiv:2312.09579*.

[56] C. Zhang et al., "Faster segment anything: Towards lightweight SAM for mobile applications," 2023, *arXiv:2306.14289*.

[57] Z. Chen, G. Fang, X. Ma, and X. Wang, "SlimSAM: 0.1% data makes segment anything slim," 2023, *arXiv:2312.05284*.

[58] E. Zhang et al., "RS-SAM: Integrating multi-scale information for enhanced remote sensing image segmentation," in *Proc. Asian Conf. Comput. Vis.*, 2024, pp. 994–1010.

[59] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.

[60] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," 2015, *arXiv:1510.00149*.

[61] H. Abdi, "Singular value decomposition (SVD) and generalized singular value decomposition," *Encyclopedia Meas. Statist.*, vol. 907, no. 912, p. 44, 2007.

[62] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50× fewer parameters and <0.5MB model size," 2016, *arXiv:1602.07360*.

[63] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "FitNets: Hints for thin deep nets," 2014, *arXiv:1412.6550*.

[64] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2736–2744.

[65] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," 2016, *arXiv:1611.06440*.

[66] Y. Wang et al., "Pruning from scratch," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 7, Apr. 2020, pp. 12273–12280.

[67] W. Chen, J. Wilson, S. Tyree, K. Weinberger, and Y. Chen, "Compressing neural networks with the hashing trick," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 2285–2294.

[68] V. Vanhoucke et al., "Improving the speed of neural networks on cpus," in *Proc. Deep Learn. Unsupervised Feature Learn. NIPS Workshop*, vol. 1, 2011, p. 4.

[69] M. Courbariaux, Y. Bengio, and J.-P. David, "Training deep neural networks with low precision multiplications," 2014, *arXiv:1412.7024*.

[70] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

[71] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 6848–6856.

[72] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1251–1258.

[73] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–10.

[74] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.

[75] J. L. Elman, "Finding structure in time," *Cognit. Sci.*, vol. 14, no. 2, pp. 179–211, Mar. 1990.

[76] J. T. H. Smith, A. Warrington, and S. W. Linderman, "Simplified state space layers for sequence modeling," 2022, *arXiv:2208.04933*.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.

[77] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023, *arXiv:2312.00752*.
[78] Y. Liu et al., "VMamba: Visual state space model," 2024, *arXiv:2403.10696*.
[79] A. Behrouz, M. Santacatterina, and R. Zabih, "MambaMixer: Efficient selective state space models with dual token and channel selection," 2024, *arXiv:2403.19888*.
[80] J. Ma, F. Li, and B. Wang, "U-mamba: Enhancing long-range dependency for biomedical image segmentation," 2024, *arXiv:2401.04722*.
[81] Y. Teng et al., "DiM: Diffusion mamba for efficient high-resolution image synthesis," 2024, *arXiv:2405.14224*.
[82] K. Li et al., "VideoMamba: State space model for efficient video understanding," 2024, *arXiv:2403.06977*.
[83] H. Zhang, K. Chen, C. Liu, H. Chen, Z. Zou, and Z. Shi, "CDMamba: Remote sensing image change detection with mamba," 2024, *arXiv:2406.04207*.
[84] K. Chen, B. Chen, C. Liu, W. Li, Z. Zou, and Z. Shi, "RSMamba: Remote sensing image classification with state space model," *IEEE Geosci. Remote Sens. Lett.*, vol. 21, pp. 1–5, 2024.
[85] C. Liu, K. Chen, B. Chen, H. Zhang, Z. Zou, and Z. Shi, "RSCaMa: Remote sensing image change captioning with state space model," *IEEE Geosci. Remote Sens. Lett.*, vol. 21, pp. 1–5, 2024.
[86] H. Chen, J. Song, C. Han, J. Xia, and N. Yokoya, "ChangeMamba: Remote sensing change detection with spatiotemporal state space model," 2024, *arXiv:2404.03425*.
[87] X. Ma, X. Zhang, and M.-O. Pun, "RS^{3}Mamba: Visual state space model for remote sensing image semantic segmentation," *IEEE Geosci. Remote Sens. Lett.*, vol. 21, pp. 1–5, 2024.
[88] X. He et al., "Pan-mamba: Effective pan-sharpening with state space model," 2024, *arXiv:2402.12192*.
[89] L. Pan et al., "MambaRSIS: Context-aware multi-scale feature aggregation with selective state space model for remote sensing instance segmentation," *Eng. Appl. Artif. Intell.*, vol. 160, Nov. 2025, Art. no. 111993.
[90] K. Chen et al., "MMDetection: Open MMLab detection toolbox and benchmark," 2019, *arXiv:1906.07155*.
[91] M. Contributors, *MMEngine: Openmmlab Foundational Library for Training Deep Learning Models*, 2022.
[92] Z. Tian, C. Shen, X. Wang, and H. Chen, "BoxInst: High-performance instance segmentation with box annotations," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 5443–5452.
[93] Z. Cai and N. Vasconcelos, "Cascade R-CNN: High quality object detection and instance segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 5, pp. 1483–1498, May 2021.
[94] Z. Liu et al., "Swin transformer V2: Scaling up capacity and resolution," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 12009–12019.
[95] T. Liang et al., "CBNet: A composite backbone network architecture for object detection," *IEEE Trans. Image Process.*, vol. 31, pp. 6893–6906, 2022.
[96] Y. Li, H. Mao, R. Girshick, and K. He, "Exploring plain vision transformer backbones for object detection," in *Proc. 17th Eur. Conf. Comput. Vis. (ECCV)*, Oct. 2022, pp. 280–296.
[97] T. Vu, H. Kang, and C. D. Yoo, "SCNet: Training inference sample consistency for instance segmentation," in *Proc. AAAI Conf. Artif. Intell.*, Feb. 2021, pp. 2701–2709.
[98] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[99] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," 2019, *arXiv:1905.11946*.
[100] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 25, 2012, pp. 1–11.
[101] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
[102] A. Howard, A. Zhmoginov, L.-C. Chen, M. Sandler, and M. Zhu, "Inverted residuals and linear bottlenecks: Mobile networks for classification, detection and segmentation," in *Proc. CVPR*, Jun. 2018, pp. 4510–4520.
[103] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2017, pp. 4700–4708.
[104] C. Szegedy et al., "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1–9.
[105] J. Dai et al., "Deformable convolutional networks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 764–773.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 03:26:47 UTC from IEEE Xplore. Restrictions apply.